client | Swing

ClientApplication

launch

LoginFrame
-host: String
-port: int

login

RegisterDialog

register

StorePanel

MainFrame
-session: Session
-navigationModel: ModuleNavigationModel
+showModule(module): void
+logout(): void

logout

CourseManagementPanel

UserManagementPanel

CourseSelectionPanel

RequestLifecycle

ModuleNavigationModel

ModuleDescriptor

RemoteUserService

RemoteStoreService

RemoteCourseService

SocketMessageClient
-socket: Socket
-output: ObjectOutputStream
-input: ObjectInputStream
+send(request: Message): Message
+close(): void

sends

library

LibraryService
+search(keyword): ServiceResult<List<Book>>
+borrow(studentId, bookId): ServiceResult<Void>
+returnBook(studentId, bookId): ServiceResult<Void>

Book
-bookId: String
-title: String
-author: String

common

Message
-requestId: String
-type: MessageType
-statusCode: StatusCode
-sender: String
-payload: Object
+request(requestId, type, payload): Message
+response(request, statusCode, payload): Message
+withSender(value): Message

MessageType
REGISTER / LOGIN / LOGOUT / UNREGISTER
AUTHORIZE / USER_LIST / USER_IMPORT
USER_ENABLE / USER_DISABLE / USER_ROLE_CHANGE
USER_AUDIT_LIST / PASSWORD_RESET_* / PASSWORD_CHANGE
STUDENT_QUERY / STUDENT_UPDATE
COURSE_QUERY / COURSE_SELECT / COURSE_DROP
COURSE_MANAGE / COURSE_SELECTION_QUERY_V2
COURSE_SELECT_OFFERING_V2 / COURSE_DROP_RECORD_V2
LIBRARY_QUERY / LIBRARY_BORROW / LIBRARY_RETURN
STORE_QUERY / STORE_PURCHASE / STORE_ORDER_QUERY
STORE_RESTOCK / STORE_PRODUCT_* / STORE_CART_*

StatusCode
OK / BAD_REQUEST / UNAUTHORIZED / FORBIDDEN
NOT_FOUND / CONFLICT / SERVER_ERROR

User
-userId: String
-displayName: String
-role: Role

response mapping

ServiceResult<T>
-status: StatusCode
-data: T
-message: String
+ok(data): ServiceResult<T>
+failure(status, message): ServiceResult<T>

Role
ADMIN / STUDENT / TEACHER
ACADEMIC_ADMIN / LIBRARIAN / STORE_MANAGER

user-management

UserAccount
-user: User
-passwordHash: String
-active: boolean
-forcePasswordChange: boolean
-createdBy: String
-createdAt: Instant
-importBatchId: String

UserCredentials
-userId: String
-password: String
-displayName: String
-roleCode: String
-profileId: String

UserRegistrationCommand
-token: String
-credentials: UserCredentials

PasswordHasher
+hash(password): String
+matches(password, encoded): boolean

Session
-token: String
-user: User
-forcePasswordChange: boolean

SessionManager
-sessions: Map<String, Session>
-activeTokensByUserId: Map<String, String>
+create(user): Session
+find(token): Session
+invalidate(token): boolean
+invalidateUser(userId): void

UserManagementService
+register(credentials): ServiceResult<Void>
+listAccounts(token): ServiceResult<List<UserAccountSummary>>
+importUsers(token, rows): ServiceResult<UserImportResult>
+setAccountActive(command): ServiceResult<Void>
+changeUserRole(command): ServiceResult<Void>
+listAuditEvents(token): ServiceResult<List<AuditEvent>>
+requestPasswordReset(command): ServiceResult<Void>
+reviewPasswordReset(command): ServiceResult<PasswordResetReviewResult>
+login(credentials): ServiceResult<Session>
+currentSession(token): ServiceResult<Session>
+logout(token): ServiceResult<Void>
+authorize(token, permission): ServiceResult<Boolean>

InMemoryUserManagementService
-delegate: UserManagementService

delegates

DefaultUserManagementService
-users: UserRepository
-sessions: SessionManager
-auditLog: AuditLogRepository
-passwordResets: PasswordResetApplicationRepository
-profileBindings: ProfileBindingRepository
-passwordHasher: PasswordHasher
-permissionPolicy: RolePermissionPolicy
+register / login / logout / authorize
+import / account status / role change
+password reset workflow

RolePermissionPolicy
-matrix: Map<Role, Set<Permission>>
+isAllowed(role, permission): boolean

Permission
USER_SELF_READ / USER_MANAGE
STUDENT_READ / STUDENT_WRITE
COURSE_READ / COURSE_SELECT / COURSE_MANAGE
GRADE_WRITE / ACADEMIC_REVIEW
LIBRARY_READ / LIBRARY_BORROW / LIBRARY_MANAGE
STORE_READ / STORE_PURCHASE / STORE_MANAGE

AuditLogRepository
+record(event): void
+findAll(): List<AuditEvent>

InMemoryAuditLogRepository

PasswordResetApplicationRepository
+save(application): void
+findPendingByUserId(userId): PasswordResetApplication
+findPending(): List<PasswordResetApplication>
+review(userId, status, reviewer, time): boolean

InMemoryPasswordResetApplicationRepository

ProfileBindingRepository
+validate(role, profileId, userId): ProfileBindingResult
+bind(role, profileId, userId): ProfileBindingResult
+findProfileId(role, userId): String

NoOpProfileBindingRepository

InMemoryProfileBindingRepository

UserRepository
+create(account): boolean
+findById(userId): UserAccount
+findAll(): List<UserAccount>
+setActive(userId, active): boolean
+changeRole(userId, role): boolean
+updatePasswordHash(userId, hash, force): boolean

InMemoryUserRepository

UserImportResult

UserRoleChangeCommand

UserStatusCommand

UserCommand
-token: String
-userId: String

UserImportCommand

PasswordResetApplication

PasswordResetRequestCommand

AuthorizationRequest

AuditEvent

UserAccountSummary

UserImportRow

PasswordResetStatus

PasswordChangeCommand

PasswordResetApplicationSummary

PasswordResetReviewResult

PasswordResetReviewCommand

ProfileBindingResult

authorize

authorize

server | Socket

AccessAuditLogRepository

AccessPasswordResetApplicationRepository

AccessProfileBindingRepository

AccessUserRepository

AccessProductRepository

AccessOrderRepository

UserMessageHandler
-service: UserManagementService
+handle(request): Message

StoreMessageHandler
-store: StoreService
-users: UserManagementService
+handle(request): Message

CourseMessageHandler
-courses: CourseSelectionService
-catalog: CourseCatalogService
-offerings: CourseOfferingService
-profiles: StudentSelectionProfileProvider
-users: UserManagementService
+handle(request): Message

ServerApplication
-port: int
-userMessages: UserMessageHandler
-courseMessages: CourseMessageHandler
-storeMessages: StoreMessageHandler
-clients: ExecutorService
+start(): void
+close(): void
-dispatch(request): Message

ClientHandler
-socket: Socket
+run(): void

bootstrap

StoreServiceFactory

UserServiceFactory

bootstrap

store

StoreProductUpdateCommand

StoreProductAddCommand

StoreRestockCommand

StoreOrderQueryCommand

StoreQueryCommand

CartQueryCommand

CartRemoveCommand

CartAddCommand

StoreProductDeactivateCommand

StoreHotProductsCommand

StoreOrderListAllCommand

CartCheckoutCommand

Order
-orderId/userId/productId: String
-quantity: int
-totalPrice/unitPrice: double
-orderDate: LocalDateTime
-productName: String <snapshot>

productId snapshot

productId / price snapshot

Product
-productId: String
-name: String
-stock: int
-price: double
-description/category: String
-active: boolean

CartItem
-cartItemId/userId/productId: String
-quantity: int
-addedAt: LocalDateTime

productId

productId

StoreService
+listProducts(): ServiceResult<List<Product>>
+purchase(userId, productId, quantity): ServiceResult<Void>
+findOrdersByUserId(userId): ServiceResult<List<Order>>
+restock(userId, productId, amount): ServiceResult<Void>
+add/update/deactivateProduct(...)
+addToCart/removeFromCart/getCart(...)
+checkout(userId): ServiceResult<Void>
+findAllOrders(): ServiceResult<List<Order>>
+listHotProducts(limit): ServiceResult<List<Product>>

StorePurchaseCommand

InMemoryStoreService

delegates

DefaultStoreService

ProductRepository
+findAll(): List<Product>
+findById(id): Product
+save/updateProduct(product): boolean
+updateStock(productId, stock): boolean
+addStock(productId, amount): boolean
+deleteById(productId): boolean

InMemoryProductRepository

CartRepository
+findByUserId(userId): List<CartItem>
+addItem/removeItem(...): boolean
+updateQuantity(itemId, quantity): boolean
+clearByUserId(userId): void

InMemoryCartRepository

OrderRepository
+create(order): boolean
+findByUserId(userId): List<Order>
+findAll(): List<Order>
+findSalesVolume(): List<Object[]>

InMemoryOrderRepository

student-management

StudentManagementService
+findById(studentId): ServiceResult<StudentRecord>
+findByClass(classId): ServiceResult<List<StudentRecord>>
+save(record): ServiceResult<StudentRecord>

studentId

StudentRecord
-studentId: String
-name: String
-gender: String
-departmentName: String
-majorName: String
-classId: String
-enrollmentYear: int
-status: String
-phone/email: String

computes

computes

AcademicReview
-studentId: String
-totalEarnedCredits: int
-passedCourseCount: int
-failedCourseCount: int
-retakeCourseCount: int
-graduationReady: boolean
-remark: String

InMemoryAcademicReviewService
-historiesByStudentId: Map<String, List<CourseHistoryRecord>>
+addHistory(record): ServiceResult<Void>
+historyFor(studentId): ServiceResult<List<CourseHistoryRecord>>
+review(studentId, requiredCredits): ServiceResult<AcademicReview>

CourseHistoryRecord
-studentId/courseId: String
-courseName/semester: String
-attemptNo: int
-score/earnedCredits: int
-attemptType: String
-passed: boolean

course-selection

profile bridge

CourseQueryCommand

ScheduleConflict

StudentSelectionProfile

SelectionRoundService

SelectionRound

TrainingPlan

CourseDropRecordV2Command

SelectedCourseOffering

SelectableCourseOffering

CourseOfferingCapacitySnapshot

CourseSelectionQueryV2Command

CourseManagementCommand
-token: String
-operation: Operation
-course/offering/targetId: Object
-teacherId/location: String
+listCourses(token): CourseManagementCommand
+createOffering(token, offering): CourseManagementCommand
+updateTeachingInfo(token, offeringId, teacherId, location): CourseManagementCommand

CourseSelectionModule

CourseSelectOfferingV2Command

CourseStatus

CourseSelectionCommand

InMemorySelectionRoundService

SelectionRoundStatus

SelectionRoundType

TrainingPlanCourse

CourseSelectionRecord

SelectionRecordStatus

SelectionType

CapacityBucket

CourseOffering
-offeringId: String
-courseId: String
-term: String
-teacherId: String
-schedule: String
-location: String
-requiredCapacity: int
-electiveCapacity: int
-crossMajorCapacity: int
-status: CourseOfferingStatus
-meetingSchedule: CourseSchedule
+getCapacity(bucket): int
+getTotalCapacity(): int

CourseOfferingStatus

CourseSchedule

CourseMeeting

Course
-courseId: String
-name: String
-credits: int
-capacity: int
-status: CourseStatus

CapacityBucketUsage

CourseCatalogService
+create(course): ServiceResult<Course>
+findById(courseId): ServiceResult<Course>
+listAll(): ServiceResult<List<Course>>
+listActive(): ServiceResult<List<Course>>
+updateDetails(courseId, name, credits): ServiceResult<Course>
+changeStatus(courseId, status): ServiceResult<Course>

InMemoryCourseCatalogService

CourseSelectionService
+listAvailableRounds(student, time): ServiceResult<List<SelectionRound>>
+listAvailableOfferings(student, roundId, time): ServiceResult<List<SelectableCourseOffering>>
+select(student, roundId, offeringId, time): ServiceResult<CourseSelectionRecord>
+drop(student, recordId, time): ServiceResult<CourseSelectionRecord>
+listSelectedOfferings(student): ServiceResult<List<SelectedCourseOffering>>

DefaultCourseSelectionService

CourseOfferingCapacityService
+snapshotFor(offeringId): ServiceResult<CourseOfferingCapacitySnapshot>

ScheduleConflictDetector

TrainingPlanService

StudentSelectionProfileProvider
+findByUserId(userId): ServiceResult<StudentSelectionProfile>

DefaultCourseOfferingCapacityService

CourseSelectionRecordService
+create(record): ServiceResult<CourseSelectionRecord>
+findById(recordId): ServiceResult<CourseSelectionRecord>
+listByStudent(studentId): ServiceResult<List<CourseSelectionRecord>>
+listActiveByOffering(offeringId): ServiceResult<List<CourseSelectionRecord>>
+markDropped(recordId, time): ServiceResult<CourseSelectionRecord>

InMemoryCourseSelectionRecordService

CourseOfferingService
+create(offering): ServiceResult<CourseOffering>
+findById(offeringId): ServiceResult<CourseOffering>
+listByTerm(term): ServiceResult<List<CourseOffering>>
+changeStatus(offeringId, status): ServiceResult<CourseOffering>
+changeCapacities(offeringId, ...): ServiceResult<CourseOffering>
+updateTeachingInfo(offeringId, teacherId, location): ServiceResult<CourseOffering>

InMemoryCourseOfferingService